

Question Bank — Week 13

Hash Tables (Lecture 13) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the code	4
Part D — Table state: draw every step	6
Part E — Complexity analysis	7
Part F — Find and fix the bug	8
Part G — Write the code	9

Questions: [week13-questions.md](#). Commit to your own answer before reading one here. Every table state, probe list and printed line below was produced by running the reference `dsa/hashmap.py` (`solutions/dsa/hashmap.py`), or the code shown, not worked out by hand.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	a	M19	b
M02	a	M08	c	M14	d	M20	d
M03	c	M09	b	M15	c	M21	a
M04	d	M10	c	M16	a	M22	d
M05	a	M11	d	M17	b		
M06	a	M12	b	M18	c		

W13-M01 — b. hash turns the key into an integer; `% capacity` folds it into `0..capacity - 1`. (a) only works for integer keys and puts all small keys in slot 0; (c) is searching, which is what hashing avoids.

W13-M02 — a. $45 = 5 \times 8 + 5$.

W13-M03 — c. Otherwise two equal keys look in different buckets, and a key that is present is reported missing. (a) is impossible — there are more keys than hash values — and not needed: collisions are handled.

W13-M04 — d. Hash randomisation, on by default since Python 3.3, so that an attacker cannot predict which strings collide. Within one run the hash never changes. (b) is how objects **without** a `__hash__` of their own are hashed, not strings.

W13-M05 — a. `hash(n)` is `n` for every integer of this size. `str` and `bytes` hashes are randomised (b, c), and a set's order follows the hashes (d).

W13-M06 — a. A mutable key could change after put; its hash would change with it, and it would sit in a bucket no lookup visits. So Python refuses to hash lists, dicts and sets at all.

W13-M07 — b. $1 - \frac{365}{365} \cdot \frac{364}{365} \dots \frac{343}{365} \approx 0.507$ — the birthday paradox. $23/365$ (c) is about 6% (a), the chance for one given pair, and ignores the other 252 pairs.

W13-M08 — c. For chaining it is also the average chain length.

W13-M09 — b. All three are $5 \bmod 8$, and each new key is pushed on the **front**: after 5, then $13 \rightarrow 5$, then $21 \rightarrow 13 \rightarrow 5$. Load $3/8$: no resize.

W13-M10 — c. $29 \bmod 8 = 5$: it compares with 21, 13 and 5, reaches the end of the chain, and raises `KeyError`.

W13-M11 — d. The check runs after the insert, with $>$: $6/8 = 0.75$ is allowed, $7/8 = 0.875$ is not. The table goes from 8 to 16 buckets during the 7th put.

W13-M12 — b. A key in bucket 2 of 8 (say 10) belongs in bucket 10 of 16. Copied across, it stays in bucket 2, and `get(10)` looks in bucket 10: `KeyError`. Every key must be rehashed.

W13-M13 — a. $6 \rightarrow$ slot 6. $14 \rightarrow 6$ is taken, slot 7. $22 \rightarrow 6$, 7 taken, wrap to slot 0. Slots: 22 _ _ _ _ 6 14.

W13-M14 — d. $30 \bmod 8 = 6$: probes 6 (6), 7 (14), 0 (22), 1 (None) — four probes, then “not there”.

W13-M15 — c. A search stops at None, because put would have used that slot. Emptying a slot in the middle of a probe sequence makes every key beyond it unreachable (`test_open_addressing_delete_preserves_probe_chain`).

W13-M16 — a. A tombstone means “a key was here once, and others may have probed past it”. Only None ends a search.

W13-M17 — b. Tombstones are not keys (a, c), but they are not None either. Uncounted, a table under many puts and deletes fills with tombstones: every miss probes the whole table, and a loop that waits for None never ends.

W13-M18 — c. A run of length L is hit by any key whose home is in it or next to it, and the key lands at its end, making it longer. (b) is not clustering in the linear-probing sense.

W13-M19 — b. $\frac{1}{2}(1 + 1/0.25) = \frac{1}{2} \cdot 5 = 2.5$. (a) is the **successful** search at 0.5; (d) the unsuccessful one at 0.9. Measured in Lecture 13 on 16,384 slots: 2.56.

W13-M20 — d. Each key is a multiple of 1,024, so $\text{key} \% \text{capacity}$ is 0 for every power-of-two capacity up to 1,024. Resizing does not help (a): the keys collide at every such capacity. One chain of n entries: $O(n)$ per get, $O(n^2)$ to build.

W13-M21 — a. CPython's `Objects/dictobject.c`: open addressing, probe $i = (5*i + 1 + \text{perturb}) \% 2**k$ with `perturb` taken from the hash, a resize at about two-thirds, and “dummy” entries as tombstones. (b) is Java's `HashMap`.

W13-M22 — d. A range query needs order, and hashing destroys order on purpose: 2027114 and 2027115 land far apart. A sorted array answers it with two `lower_bounds` (Week 8). (a)–(c) are $O(1)$ average in a hash table.

Part B — Short answer and essay

W13-E1 (4)

- **Mechanism:** the index $\text{hash}(\text{key}) \% \text{capacity}$ is **computed** in $O(1)$ and the Array slot is reached in $O(1)$; only the keys that share that slot are compared — a chain, or a short run of probes.
- **Assumption:** the hash spreads the keys evenly, as if at random, and the load factor α is kept below a constant by resizing. Then a chain averages α entries (a probe run: a few slots), so get, put and delete are $O(1)$ **expected**, and put amortised because of resizes.
- **Worst case:** all n keys in one bucket or one cluster: $O(n)$ per operation.
- **What pushes towards it** (any three): a poor hash (sum of characters — anagrams collide); keys sharing their low bits on a power-of-two table (multiples of 1,024); an adversary who can predict the hash (hashDoS — hence string hash randomisation); no resizing, so α grows with n ; open addressing near $\alpha = 1$, or clogged with tombstones.

W13-E2 (3)

- **Only slow — never wrong.** Every key hashes to 0 and lands in bucket (or home slot) 0; keys are still told apart by `==`, so every get finds the right entry or correctly reports it missing.
- **Chaining:** one chain holding all n keys. get walks it: $O(n)$. Each put walks it too (to check for the key): building n keys is $1 + 2 + \dots + n = O(n^2)$. Resizing never helps: 0 % capacity is always 0.
- **Open addressing:** all keys form one cluster starting at slot 0; a get probes up to n slots, $O(n)$; building is $O(n^2)$ — the same.
- The only hash rule for correctness is “equal keys, equal hashes”, and a constant keeps it. Every other quality of a hash function is about speed.

W13-E3 (4)

- **A slot holds:** chaining — None or the head of a linked list of Entry nodes; open addressing — one key (in `_keys`) and its value (in `_values`), or None, or a tombstone.

- **Load factor:** chaining still works above 1 (chains simply grow; cost about $1 + \alpha$), and resizes at 0.75 here; open addressing cannot exceed 1 at all and degrades sharply near it (a miss $\approx \frac{1}{2}(1 + 1/(1 - \alpha)^2)$), so it resizes earlier — 0.66 here, about 2/3 in CPython.
- **Delete:** chaining unlinks a node (the head is the special case); open addressing must leave a tombstone, which later puts may reuse and a resize drops.
- **Advantages:** chaining — simple deletion, graceful at high load, never “full”; open addressing — no node per key (less memory, no allocation), and consecutive probes are neighbours in memory, which suits the cache. CPython’s dict and set use open addressing; Java’s HashMap uses chaining.

W13-E4 (4)

- **Why not empty the slot:** a search stops at None. If keys a and b collide and b was placed beyond a, emptying a’s slot cuts b’s probe sequence: get (b) stops at the new None and reports b missing although it is still in the table.
- **get / delete:** skip a tombstone and keep probing; stop only at the key or at None.
- **put:** remember the first tombstone passed, but keep probing to None — the key may be further on (then overwrite it). If it is not there, insert at the remembered tombstone (reusing the space), else at the None.
- **_resize:** re-insert only live keys; tombstones are dropped, so the new table has none.
- **Load check:** tombstones are not keys, but they fill slots, and only None stops a probe. Counting them (size + tombstones + 1) forces a rebuild — at the same capacity if the live keys alone are within the limit — before the table runs out of None.

W13-E5 (3)

- A resize at size s rehashes s keys. With doubling, the resizes before reaching size n happened at sizes of at most $n, n/2, n/4, \dots$, so the total rehashing is at most $n + n/2 + n/4 + \dots < 2n$: $O(n)$ for n puts, **amortised** $O(1)$ each. (Measured in Lecture 13: 3,075 keys rehashed for 3,000 puts in the chaining table, 5,396 in the open-addressing one — both under $2n$.)
- Growing by 8 slots: a resize every 6 or so puts, each rehashing all keys so far: $6 + 12 + 18 + \dots \approx n^2/12 = \Theta(n^2)$ in total, $\Theta(n)$ amortised per put. The same argument as the DynamicArray with a constant growth step (Lecture 04).

Part C — Trace the code**W13-T1**

Key	% 8	% 16
7	7	7
15	7	15
23	7	7
-9	7	7

Key	% 8	% 16
40	0	8
100	4	4

Python's % is non-negative here: $-9 = -2 \times 8 + 7 = -1 \times 16 + 7$. At capacity 8, 7, 15, 23 and -9 share slot 7. At 16, 15 moves to 15, but 7, 23 and -9 still share slot 7: they differ by multiples of 16. Doubling the capacity separates only keys that differ in the next bit.

W13-T2

Operation	Chains	size	capacity	load
put 5	1: 5	1	4	0.25
put 9	1: 9 → 5	2	4	0.5
put 13	1: 13 → 9 → 5	3	4	0.75
put 2	1: 9 · 2: 2 · 5: 5 → 13	4	8	0.5
delete 9	2: 2 · 5: 5 → 13	3	8	0.375

put 2 is pushed into bucket 2, making $4/4 = 1.0 > 0.75$, so the table resizes to 8. The resize walks old bucket 1 front to back — 13, 9, 5 — pushing each on the front of its new chain: 13 → bucket 5, 9 → bucket 1, 5 → bucket 5 in front of 13. Then old bucket 2: 2 → bucket 2. delete 9 removes the only entry of bucket 1, which becomes None.

W13-T3

Operation	Probes	Slots after
put 3	3	— — — 3 — — — —
put 11	3, 4	— — — 3 11 — — —
put 19	3, 4, 5	— — — 3 11 19 — —
delete 11	3, 4	— — — 3 † 19 — —
get 19	3, 4 (†: go on), 5 → 19	unchanged
put 27	3, 4 (†: remember), 5, 6 (None) → into slot 4	— — — 3 27 19 — —

put 27 passes its load check: $(2 + 1 + 1) / 8 = 0.5$. It must probe on past the tombstone to slot 6 to be sure 27 is not already there, then goes back to the tombstone. The tombstone count returns to 0.

W13-T4

```
cat 0 6
act 0 2
tac 0 6
dog 2 4
god 2 4
```

sum_hash: cat, act and tac all sum to 312 (0 mod 8); dog and god to 314 (2 mod 8). They collide **because** they are anagrams, at every capacity — the hash ignores the order of the letters. poly_hash gives five different full hashes (98262, 96402, 114582, 99644, 102524), and the collisions left — cat with tac, dog with god — come only from folding them into 8 slots (3 bits). At another capacity they would be different pairs, or none. Collisions in a table of 8 slots are certain; collisions built into the hash function are a flaw.

W13-T5

Operation	Probes	Result	Slots after
put 0, 8, 16, 24	1, 2, 3 and 4 probes		0 8 16 24 _ _ _ _
get 24	0, 1, 2, 3	24	
get 32	0, 1, 2, 3, 4 (None)	None	
delete 8	0, 1		0 † 16 24 _ _ _ _
get 24	0, 1 (†), 2, 3	24	
put 40	0, 1 (†: remember), 2, 3, 4 (None) → slot 1		0 40 16 24 _ _ _ _
get 32	0, 1, 2, 3, 4	None	

put 40: load check $(3 + 1 + 1) / 8 = 0.625$, fine. Every key here has home 0, so they form one run, and the cost of each operation is its position in the run: the fourth key costs 4 probes, a miss 5. The tombstone neither shortens nor lengthens the run — which is exactly its job. With n such keys, a miss costs $n + 1$ probes: the $O(n)$ worst case, built from only 4 keys.

Part D — Table state: draw every step**W13-S1**

Operation	Buckets (non-empty)	size	capacity
put 10	2: 10	1	4
put 3	2: 10 · 3: 3	2	4
put 14	2: 14 → 10 · 3: 3	3	4
put 7	2: 10 · 3: 3 · 6: 14 · 7: 7	4	8
put 18	2: 18 → 10 · 3: 3 · 6: 14 · 7: 7	5	8
delete 10	2: 18 · 3: 3 · 6: 14 · 7: 7	4	8
put 26	2: 26 → 18 · 3: 3 · 6: 14 · 7: 7	5	8
delete 26	2: 18 · 3: 3 · 6: 14 · 7: 7	4	8

put 7 goes into bucket 3 ($7 \% 4$) in front of 3, making $4/4 = 1.0 > 0.75$: the table doubles. The resize walks old bucket 2 (14, then 10) and old bucket 3 (7, then 3): $14 \rightarrow 6$, $10 \rightarrow 2$, $7 \rightarrow 7$, $3 \rightarrow 3$. delete 10 unlinks the **last** entry of a chain (`prev.next = None`); delete 26 unlinks the **head** (the bucket slot itself changes). Loads after each: 0.25, 0.5, 0.75, 0.5, 0.625, 0.5, 0.625, 0.5.

W13-S2

Operation	Probes	Slots	size	†
put 5	5	— — — — — 5 — —	1	0
put 13	5, 6	— — — — — 5 13 —	2	0
put 21	5, 6, 7	— — — — — 5 13 21	3	0
put 6	6, 7, 0	6 — — — — 5 13 21	4	0
delete 13	5, 6	6 — — — — 5 † 21	3	1
put 29	5, 6 (†), 7, 0, 1 → slot 6	6 — — — — 5 29 21	4	0
delete 5	5	6 — — — — † 29 21	3	1
put 14	6, 7, 0, 1 → slot 1	6 14 — — — † 29 21	4	1

Load checks: put 6 $(3 + 0 + 1) / 8 = 0.5$; put 29 $(3 + 1 + 1) / 8 = 0.625$; put 14 $(3 + 1 + 1) / 8 = 0.625$ — none above 0.66. Note put 6: its home slot 6 is taken by 13, a key from **another** home — clustering. And put 14 never passes the tombstone in slot 5 (its sequence starts at 6), so it cannot reuse it: the tombstone stays until a probe for a home-5 key reuses it or a rebuild drops it.

W13-S3

Operation	Load check (before the insert)	Action	Slots after	size	†
put 1 ... put 5	at most $(4 + 0 + 1) / 8 = 0.625$	—	— 1 2 3 4 5 — —	5	0
delete 1			— † 2 3 4 5 — —	4	1
delete 2			— † † 3 4 5 — —	3	2
put 9	$(3 + 2 + 1) / 8 = 0.75 > 0.66$; live $(3 + 1) / 8 = 0.5$	sweep , capacity 8	— 9 — 3 4 5 — —	4	0
put 10	$(4 + 0 + 1) / 8 = 0.625$	nothing	— 9 10 3 4 5 — —	5	0
put 11	$(5 + 0 + 1) / 8 = 0.75$; live $0.75 > 0.66$	double , to 16	slots 3, 4, 5, 9, 10, 11 of 16	6	0

put 9: the sweep re-inserts 3, 4 and 5 into a clean table of 8, and then 9 (home 1) finds slot 1 empty — the tombstones are gone. put 10: home 2, empty. put 11: the table doubles to 16, re-inserting 3, 4, 5, 9, 10 at their homes in 16 slots (all distinct), then 11 at slot 11.

Part E — Complexity analysis

W13-K1 — average $\Theta(n + m)$, **worst** $\Theta(nm)$. n puts and m lookups, each $O(1)$ on average (the puts amortised over the resizes). In the worst case — every key in one chain — a put or lookup walks up to n entries: the puts alone are $\Theta(n^2)$, and the lookups $\Theta(nm)$; in all $\Theta(n^2 + nm)$. With a **list**, in is a linear scan: $\Theta(nm)$ in the worst case (every lookup a miss), whatever the keys.

W13-K2.

- **Never resizes:** the i -th put walks a chain of about $i/8$ entries to check for the key: $\sum i/8 \approx n^2/16 = \Theta(n^2)$. The table is 8 linked lists.
- **Doubles at 0.75:** chains stay at most about 0.75 long on average, so each put is $O(1)$ plus rehashing that totals less than $2n$: $\Theta(n)$.
- **Adds 8 at 0.75:** a resize every 6 puts, each rehashing everything so far: $\Theta(n^2)$ in total — the same trap as the constant growth step of Lecture 04.

W13-K3.

- **(a)** n scans of n words: $\Theta(n^2)$.
- **(b)** sorting $O(n \log n)$ comparisons (each comparing words), then one pass over the runs: $O(n \log n)$ — and the counts come out in alphabetical order, a bonus if you need it.
- **(c)** one pass; each word is one get and one put on average $O(1)$: $O(n)$ **expected**. Worst case — every word in one bucket — $O(n^2)$.
- **Use (c)**, with the language's own hash table (`dict`, or `collections.Counter`): its string hashes are randomised, so an adversary cannot force the worst case. Use (b) if the output must be sorted anyway.

Part F — Find and fix the bug

W13-B1. It treats a key whose **value** is `None` as absent. After `m.put("k", None)`, `"k" in m` is `False` although `len(m)` counts it — the code ran, and printed `False True 2` for `"k"`, `"z"` (value 0) and `len`. `None` is a legal value, so it cannot also mean “missing”. **Fix:** ask the question that is really meant — does `get` raise?

```
def __contains__(self, key):
    try:
        self.get(key)
    except KeyError:
        return False
    return True
```

(This is the version given in `dsa/hashmap.py`. The alternative is a private sentinel object as the default, which no caller can ever store.)

W13-B2. `entry = entry.next` runs **after** `entry.next` has been overwritten with the head of the new chain, so the walk jumps into the new table and the rest of the old chain is lost. On a `ChainingHashMap(capacity=4)` with `put 1, 5, 9, 2`: bucket 1 holds `9 → 5 → 1` when the 4th put resizes; 9 moves to bucket 1 of 8 with `next = None`, the loop ends, and 5 and 1 are gone. `get(1)` and `get(5)` raise `KeyError`, while `len(m)` still says 4. **Fix:** save the next entry first:

```
while entry is not None:
    nxt = entry.next
```

```

i = self._index(entry.key)
entry.next = self._buckets[i]
self._buckets[i] = entry
entry = next

```

W13-B3. It inserts at the **first** tombstone without checking whether the key is further along its probe sequence. Run: `put(0, "a")`, `put(8, "b")` (slot 1), `delete(0)` (slot 0 becomes `t`), `put(8, "B")` — the key 8 is written into slot 0, although it is already in slot 1. `len(m)` is 2, `list(m)` is `[8, 8]`, and after `delete(8)` the table still answers `get(8)` with the old "b". It also forgets `_tombstones -= 1` when it reuses one. **Fix:** remember the first tombstone, keep probing until the key or `None`, and only then insert (Lecture 13, the tombstone table; the reference put).

W13-B4. `self._buckets[i] = entry.next` is right only when the match is the **head** of its chain. For a match further in, it makes the bucket start after the match, dropping every entry **before** it. With 3, 11, 19 in one chain (`19 → 11 → 3`), `delete(11)` leaves bucket 3 as just 3: `len(m)` says 2, but 19 is gone — `get(19)` raises `KeyError`. The graded tests only delete from a one-key table, so they pass. **Fix:** walk with `prev`, and unlink with `prev.next = entry.next` unless `prev` is `None`:

```

def delete(self, key):
    i = self._index(key)
    prev, entry = None, self._buckets[i]
    while entry is not None:
        if entry.key == key:
            if prev is None:
                self._buckets[i] = entry.next
            else:
                prev.next = entry.next
            self._size -= 1
            return
        prev, entry = entry, entry.next
    raise KeyError(key)

```

Part G — Write the code

Every solution here is the code in `solutions/practice/week13.py`, line for line, and passes `pytest --solutions tests/test_practice_week13.py`. Each uses a `ChainingHashMap` as its working storage.

W13-C1

```

def two_sum(values, target):
    """W13-C1. Indices (i, j), i < j, with values[i] + values[j] == target."""
    first_seen = ChainingHashMap()          # value -> first index seen

```

```

for j, value in enumerate(values):
    i = first_seen.get(target - value, None)
    if i is not None:
        return (i, j)
    if value not in first_seen:
        first_seen.put(value, j)
return None
    
```

One pass. For each value, the partner it needs is $\text{target} - \text{value}$; if that partner has already been seen, its **first** index is the smallest i for this j , and this j is the smallest j with any partner — so the first pair found is the one the docstring asks for. Store a value only the **first** time it is seen: overwriting it with later indices would make $[3, 3, 5]$ with target 8 return $(1, 2)$ instead of $(0, 2)$. $O(n)$ expected, against $O(n^2)$ for trying every pair.

W13-C2

```

def first_repeated(items):
    """W13-C2. The first item to appear for the second time, or None."""
    seen = ChainingHashMap() # used as a set: item -> True
    for item in items:
        if item in seen:
            return item
        seen.put(item, True)
    return None
    
```

The map is used as a **set** — only its keys matter. Returning the moment an item is found again gives the item whose second occurrence is earliest. $O(n)$ expected; the obvious “for each item, look back through the earlier ones” is $O(n^2)$.

W13-C3

```

def group_anagrams(words):
    """W13-C3. Group the words that are anagrams of one another."""
    groups = []
    where = ChainingHashMap() # sorted letters -> group index
    for word in words:
        key = "".join(sorted(word))
        i = where.get(key, None)
        if i is None:
            where.put(key, len(groups))
            groups.append([word])
        else:
            groups[i].append(word)
    return groups
    
```

Two words are anagrams exactly when their sorted letters are equal, so `"".join(sorted(word))` is the key (a string: hashable, unlike the list `sorted` returns). The map sends the key to the **position** of its group in the result, so the groups come out in order of first appearance,

and each group grows in input order. The result list is output, which the storage rule allows. Sorting each word costs $O(k \log k)$: $O(nk \log k)$ in all.

W13-C4

```
def longest_distinct_run(text):
    """W13-C4. The longest stretch of text with no repeated character."""
    last_seen = ChainingHashMap()           # character -> last index
    start = best = 0
    for i, ch in enumerate(text):
        previous = last_seen.get(ch, -1)
        if previous >= start:                # ch repeats inside the window
            start = previous + 1
        last_seen.put(ch, i)
        best = max(best, i - start + 1)
    return best
```

Why $O(n)$: the window `text[start..i]` never contains a repeat. When `ch` was last seen **inside** the window (at `previous >= start`), the window must start just after that position; `start` only ever moves **forward**, so the two ends of the window each move at most n times. The map remembers each character's last position so the jump is $O(1)$, with no scan. The test `previous >= start` matters: a character last seen **before** the window (in "abba", the second a) must not drag `start` back.

W13-C5

```
def count_subarrays_with_sum(values, k):
    """W13-C5. How many contiguous runs of values add up to k?"""
    seen = ChainingHashMap()                # prefix sum -> times seen
    seen.put(0, 1)                          # the empty prefix
    total = count = 0
    for value in values:
        total += value
        count += seen.get(total - k, 0)
        seen.put(total, seen.get(total, 0) + 1)
    return count
```

Why not a sliding window: with negative values, extending a window can make its sum smaller, so "too big — shrink it" is no longer a safe rule. **Why prefix sums:** `values[i:j]` adds up to `prefix[j] - prefix[i]`. So at each j , the runs ending there that sum to k are exactly the earlier prefixes equal to `prefix[j] - k`; the map counts how many times each prefix sum has occurred (starting with the empty prefix, 0, once). One pass: $O(n)$ expected. $[0] * n$ with $k = 0$ has $n(n+1)/2$ runs — 200,010,000 for $n = 20,000$ — so no method that lists them one by one can be fast.